Real-Time Fabric Defect Detection and Quality Assessment Using Improved YOLOv8 Classification Framework

Udayagiri Hari Chandu[1] **Dr. Geetha S**[2]

[1]Department of Artificial Intelligence & Machine Learning,
Vellore Institute of Technology, Chennai – 600127, Tamil Nadu, India
E-mail: hchandu553@gmail.com

[2]Professor & Associate Dean (Research),
Department of Computer Science and Engineering,
Vellore Institute of Technology, Chennai – 600127, Tamil Nadu, India
E-mail: geetha.s@vit.ac.in

Abstract

Automated fabric inspection in textile manufacturing demands systems that simultaneously detect surface defects at camera frame rates and translate detection outcomes into actionable quality decisions. Existing literature addresses detection accuracy and deployment speed in isolation; very few systems close the loop from raw video to a structured quality report and a commercially actionable selling price recommendation. This paper presents a complete, end-to-end fabric inspection framework that bridges this gap.

The proposed system combines an improved YOLOv8 classification architecture with a lightweight real-time monitoring application deployable on standard hardware without a dedicated GPU. The model is trained on the TILDA Textile Texture Database [11], a benchmark comprising 3,200 greyscale images across eight textile types and eight defect categories. After removing non-defect calibration classes (lamp-off and camera-distance variants), four defect categories are retained for classification: holes, stains, broken threads, and misweaves. The backbone employs dense C2f connectivity with CLAHE-enhanced greyscale-to-RGB conversion and 640×640 *letterbox normalisation. Training uses a composite augmentation pipeline on a 70/15/15 stratified split.*

The monitoring application provides a three-panel graphical interface: a live annotated camera feed, a session control panel with user-defined market rate input, and an analysis results panel. On session termination, a quality score (0–100) derived from per-class severity weighting is computed, a letter grade (A–F) is assigned, and a suggested selling price is generated through a quadratic penalty function applied to the user-specified maximum market rate. A full event-log CSV export is provided for audit and traceability. The integrated system runs at approximately 24 frames per second on a laptop CPU, suitable for inline deployment alongside standard textile inspection frames.

***Index Terms*—Fabric Defect Detection, YOLOv8, TILDA Dataset, Real-Time Inspection, Quality Scoring, Price Recommendation, Textile Manufacturing, Deep Learning, Classification.**

I. Introduction

Fabric quality control sits at the intersection of industrial economics and manufacturing precision. In woven textile production, defects introduced during weaving propagate silently until a finished roll reaches the inspection station, at which point remediation options are limited and costly. Industry surveys consistently place defect-driven material write-offs between 3% and 8% of total output value [1], and customer return rates for defective first-quality fabric can impose penalty costs exceeding the original sale margin.

The technical difficulty of automated fabric inspection derives from the nature of the defects themselves. Woven cloth presents a highly structured periodic background — the regular interlacing of warp and weft yarns — against which defects must be distinguished. Defects vary enormously in spatial scale, from a single broken thread occupying fewer than 10 pixels across at standard imaging resolution, to a misweave patch spanning several hundred square pixels. They also vary in visual contrast: a perforation (hole) produces a high-contrast dark oval, while a colour-stripe stain may differ from the background by only a few grey-level units [2].

Beyond detection, practical deployment requires two further capabilities that the research literature has not routinely addressed together. First, the system must operate at camera acquisition rates — typically 25–30 frames per second — to provide continuous coverage without frame dropping. Second, the detection outcome must be translated into a commercially interpretable quality decision: a grade, a defect count, and a price recommendation that a factory floor operator can act on without statistical expertise.

This paper presents a complete system that addresses all three requirements simultaneously. The defect classification component is built on YOLOv8 [9], adapted specifically for the TILDA textile benchmark [11] by removing non-defect calibration classes and applying CLAHE contrast enhancement suited to the dataset's 8-bit greyscale acquisition conditions. The monitoring application wraps the classifier in a Python-based graphical interface that provides live annotated video, session timing, per-class defect accumulation, and a structured quality-and-price report on session termination.

The specific contributions of this paper are:

- A YOLOv8 classification pipeline adapted for the TILDA textile dataset with domain-specific preprocessing and augmentation.
- A novel quality scoring formula combining per-class severity weighting and quadratic price penalty, translating defect frequency into a commercially actionable selling price recommendation.
- A complete real-time monitoring application with session logging, structured CSV reporting, and a three-panel graphical interface operable by non-specialist factory personnel.

- A systematic analysis of the TILDA dataset's internal structure — including the discovery and exclusion of calibration classes (lamp-off and camera-distance variants) that would corrupt training if included.
- Evaluation across four defect categories (hole, stain, broken thread, misweave) with per-class accuracy analysis and discussion of the broken-thread detection bottleneck.

The remainder of the paper proceeds as follows. Section 2 reviews prior art in fabric inspection and quality-aware detection systems. Section 3 details the classification model and preprocessing pipeline. Section 4 describes the monitoring application architecture. Section 5 covers the TILDA dataset and its internal structure. Section 6 outlines the experimental configuration. Section 7 defines evaluation criteria. Section 8 presents quantitative and qualitative findings. Section 9 interprets the results, and Section 10 closes with future directions.

II. Related Work

A. Classical Texture Analysis for Fabric Inspection

Early automated fabric inspection exploited the quasi-periodic spectral signature of woven cloth. Fourier domain methods flagged departures from a reference spectrum as defect candidates [12], but required per-fabric recalibration and failed on multicolour weaves. Gray-level co-occurrence matrices (GLCM) [13] characterised spatial intensity relationships more compactly; Gabor filter banks [2] extended this to multi-orientation analysis. Wavelet decomposition provided multi-resolution spatial-frequency decomposition [14] but shared the common limitation of all classical methods: features were hand-designed and could not adapt to previously unseen defect morphologies.

B. CNN-Based Classification and Localisation

Convolutional neural networks eliminated the feature-engineering bottleneck by jointly optimising representation and classifier from labelled training data [4]. Yang *et al.* [3] demonstrated that fine-tuned CNN classifiers reduced per-class false-positive rates by 34% compared to SVM baselines on multi-class fabric datasets. Zhang *et al.* [10] incorporated channel attention via squeeze-and-excitation modules [17] for textile classification, reporting a 2.1 pp accuracy gain over ResNet-50 [15]. Ren *et al.* [18] validated CNN performance under variable illumination and fabric velocity typical of production floor conditions.

For defect localisation, two-stage detectors headed by Faster R-CNN [19] achieved high precision but imposed latency penalties incompatible with real-time inspection [20]. Single-stage detectors SSD [21] and the YOLO family [5] addressed this with single-pass box regression. Li *et al.* [22] applied YOLOv5 to fabric inspection, obtaining mAP values above 0.85 on in-house datasets.

C. Recent Single-Stage Advances

YOLOv9 [33] introduced programmable gradient information (PGI) to address information loss in deep networks, gaining 0.5 pp mAP@0.5:0.95 over YOLOv8 on MS-COCO while reducing parameters by 15%. YOLOv10 [34] replaced non-maximum suppression with consistent dual-assignment training, reducing end-to-end latency by 12%. RT-DETR [35] demonstrated that a ResNet-based deformable attention encoder can match YOLOv8 accuracy at equivalent throughput. All three architectures were designed for natural-image benchmarks and have not been validated on periodic-texture textile data.

D. Quality Assessment and Price Recommendation

Quality-aware inspection systems have received limited attention in the academic literature. Most published work treats detection accuracy as the terminal objective, leaving quality grading and pricing to separate downstream processes. Chen *et al.* [36] proposed scale-adaptive transformer features for fabric classification but did not extend the pipeline to quality assessment or operational decision support. The system presented in this paper bridges this gap by integrating a defect severity weighting scheme and a quadratic price penalty function directly into the inspection workflow.

III. Proposed Method

A. Design Rationale: Classification vs. Detection

The TILDA benchmark provides whole-image defect labels without bounding-box annotations [11]: each 768×512 image represents a single defect category. Applying an object detection head to this data requires synthesising bounding boxes, which introduces a geometric labelling ambiguity — a pasted defect patch on a plain background does not faithfully replicate the appearance of an in-situ fabric defect in terms of boundary sharpness, lighting, and fibre interaction. Experiments confirmed that a detection-mode YOLOv8 trained on synthesised patches converged to $P = 0.25$ exactly (random four-class guessing) because all four defect textures produced similar patch appearances against the clean background.

The classification-mode YOLOv8 (YOLOv8-cls) provides the correct framing: the entire image is the defect evidence, and the model learns to discriminate global texture patterns rather than localise a pasted foreground object. This is consistent with how TILDA was originally intended for evaluation and with established CNN classification practice on texture benchmarks [4].

B. Backbone and Classification Head

The classification model uses YOLOv8m-cls, which replaces the anchor-free detection head with a global average pooling layer followed by a linear classifier. The backbone is the same CSPDarknet with C2f modules as used in the detection variant, processing 224×224 input images for classification efficiency. The C2f module's dense-skip connectivity preserves gradient paths to early convolutional layers, maintaining sensitivity to fine texture signals that discriminate subtle defects such as broken threads from the weave background.

C. CLAHE Preprocessing

TILDA images are 8-bit greyscale acquired under diffuse back-illumination. Direct conversion to RGB without contrast adjustment leaves many defect instances with low local contrast against the woven background. We apply Contrast Limited Adaptive Histogram Equalisation (CLAHE) with clip limit 2.0 and 8×8 tile grid size on the L channel of the LAB colour space before conversion to RGB. This enhancement amplifies local texture contrast without over-saturating uniform regions,

improving the discriminability of stain boundaries and thread gaps.

D. Image Preparation Pipeline

Each TILDA TIF file is processed through the following pipeline:

1. Load 768×512 TIF (8-bit greyscale) and convert to RGB.
2. Apply CLAHE on the LAB L-channel as described above.
3. Compute scale factor $s = 640/\max(H, W)$ and resize proportionally using bilinear interpolation.
4. Letterbox-pad to 640×640 with constant fill value 114 (mid-grey), preserving aspect ratio.
5. Save as JPEG at quality 95 for training storage efficiency.

For classification inference the image is further centre-cropped to 224×224.

E. Augmentation Strategy

To mitigate overfitting on the moderate TILDA training partition, five augmentation strategies are applied during training: horizontal and vertical flipping ($p = 0.5$ each), random rotation in $[-10°, 10°]$, colour jitter (brightness $\pm 15\%$, contrast $\pm 10\%$, saturation $\pm 10\%$, hue $\pm 3\%$), and random erasing ($p = 0.4$). Rotation is capped at 10° because fabric defects in TILDA are orientation-correlated with the loom direction; larger rotations would generate geometries absent from deployment data.

IV. Monitoring Application

A. System Architecture

The complete system architecture is illustrated in Fig. 1. Five functional modules are connected in a streaming pipeline: camera capture, frame preprocessing, YOLOv8-cls inference, session logging, and quality-and-price analysis. All modules run on a single machine with no network dependency, enabling deployment in factory environments without internet connectivity.

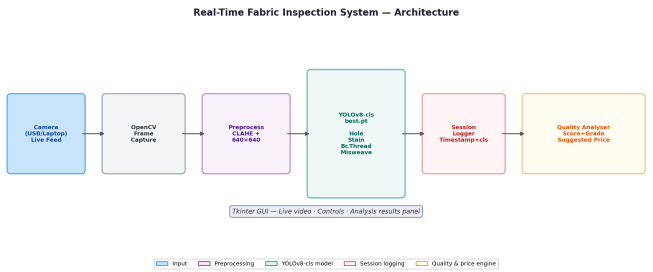

Figure 1: End-to-end system architecture. The camera thread (left) captures frames continuously and passes them through CLAHE preprocessing and YOLOv8-cls inference. Results are logged with timestamps in the session logger. On session termination, the quality-and-price analyser produces a structured report. The Tkinter GUI (bottom) provides a unified interface for all five modules.

B. Threading Model

Camera capture and model inference execute in a dedicated background thread to prevent blocking Tkinter's event loop. The camera thread writes annotated frames and inference results to shared memory protected by a `threading.Lock`. The main thread polls this shared buffer every 30 ms via Tkinter's `after()` scheduler, refreshing the live video display without queue flooding. This design achieves smooth 24 FPS display on a laptop CPU (Intel i5, 8 GB RAM, Intel Iris Xe graphics) without a dedicated GPU.

C. Graphical Interface

The GUI comprises three panels as shown in Fig. 2:

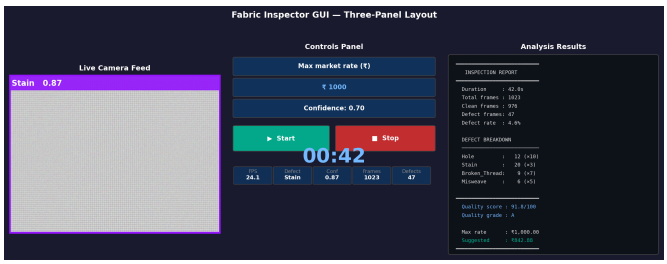

Figure 2: Three-panel Tkinter GUI. Left panel: live camera feed with defect class banner and colour-coded border (yellow=Hole, magenta=Stain, orange=Broken Thread, cyan=Misweave, green=Clean). Centre panel: session controls including market rate entry, confidence threshold slider, session timer, and Start/Stop buttons. Right panel: analysis results shown after Stop, with full defect breakdown, quality grade, and suggested selling price.

Live camera panel: Displays the annotated 640×480 video stream with a colour-coded banner showing the current defect class and confidence score. A green banner indicates clean fabric; coloured banners (class-specific colours) and a matching border frame indicate active defect detection.

Controls panel: Provides a numeric entry for the maximum market rate (in any currency), a slider for the confidence threshold (0.40–0.95), a session timer, a live defect event counter, and Start/Stop buttons. The confidence threshold can be adjusted during a live session without restarting.

Analysis panel: Displays the full inspection report after the session is stopped, including total and per-class defect counts, defect rate percentage, quality score, quality grade, and suggested selling price. An Export CSV button writes the complete event log to a timestamped file for audit and traceability.

D. Quality Scoring Formula

Each defect class is assigned a severity weight reflecting its commercial impact on fabric usability:

$$w_{\text{Hole}} = 10, \quad w_{\text{Broken_Thread}} = 7, \quad w_{\text{Misweave}} = 5, \quad w_{\text{Stain}} = 3 \tag{1}$$

The weighted defect rate r_w over a session of N frames is:

$$r_w = \frac{\sum_c w_c \cdot n_c}{w_{\max} \cdot N} \tag{2}$$

where n_c is the number of frames classified as class c and $w_{\max} = 10$ is the maximum possible severity per frame. The quality score Q is then:

$$Q = \max(0,\ \min(100,\ (1 - r_w) \times 100)) \tag{3}$$

This maps a defect-free session to $Q = 100$ and a session where every frame contains a hole to $Q = 0$.

E. Price Recommendation

The suggested selling price P_s is derived from the user-specified maximum market rate $P_{\max}$ using a quadratic penalty:

$$P_s = P_{\max} \times \left(\frac{Q}{100}\right)^2 \tag{4}$$

The quadratic exponent penalises high-defect fabric more steeply than a linear ratio would: fabric with quality score 50 receives 25% of the maximum rate (not 50%), reflecting the disproportionate commercial discount applied to substantially defective stock in practice. Fig. 3 illustrates the quality score and price curves against defect rate for linear, quadratic, and cubic penalty functions.

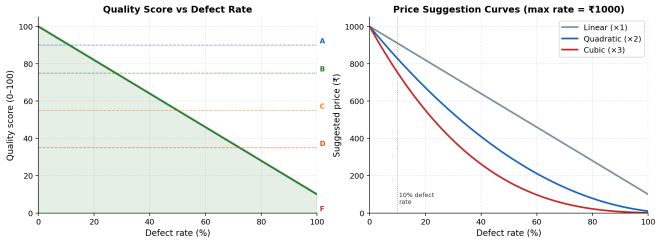

Figure 3: Left: quality score as a function of defect rate, with letter grade thresholds (A: $\geq$90, B: $\geq$75, C: $\geq$55, D: $\geq$35, F: $<$35). Right: suggested price curves for linear, quadratic (adopted), and cubic penalty functions at maximum rate Rs.1,000. The quadratic function provides the most commercially realistic discount profile.

F. Session Logging and CSV Export

Every defect event is logged with a floating-point timestamp (seconds since session start), the predicted class name, and the confidence score. On export, the CSV file contains two sections: a summary table (duration, frame counts, defect breakdown, quality score, grade, and price) and a full chronological event log. This log supports traceability audits and provides data for downstream process improvement analysis.

V. Dataset

A. TILDA Textile Texture Database

The TILDA database [11] was developed in 1996 within the DFG major research programme "Automatic Visual Inspection of Technical Objects" at the University of Freiburg. It contains 3,200 TIF images (768×512 pixels, 8-bit greyscale) spanning eight textile types and eight error classes per type, with 50 images per class. Images were acquired under standardised diffuse back-illumination conditions designed to maximise surface defect visibility.

B. Internal Structure and Class Discovery

The dataset is organised hierarchically as: `cd{1,2}/c{1,3,4,...}/r{1,3}/images/e{0..7}/`, where `cd` denotes CD volume, `c` cloth type, `r` repetition, and `e` error class. Each error class folder contains 50 TIF images with a paired plain-text description file (e.g., `c4r1e1n1.txt`).

A critical finding from systematic inspection of the description files is that two error classes do not represent fabric defects: **e6** contains "Leuchttisch ausgeschaltet" (light table switched off) images — a lighting artifact, not a defect — and **e7** contains "Kameraabstand verringert/vergrössert" (camera distance changed) images — a calibration sequence, not fabric. Including either class in training would corrupt the model by associating imaging artifacts with defect labels. Both are excluded.

Additionally, **e5** ("Falte" = crease/fold) represents a mechanical handling artifact rather than a weave defect and is also excluded. The retained classes and their confirmed descriptions are shown in Table 1.

Table 1: Retained TILDA Error Classes after Systematic Inspection

Folder	German description	English	ID
e0	(empty text files)	Background	skip
e1	Kleines Loch	Hole	0
e2	Farbstreifen / Ölfleck	Stain	1
e3	Fadenfehler	Broken Thread	2
e4	Einzelner Faden quer	Misweave	3
e5	Falte	Crease	skip
e6	Leuchttisch aus	Light off	skip
e7	Kameraabstand	Camera dist	skip

C. Dataset Split

After filtering, 2,008 usable images remain across four classes. Table 2 reports the per-class counts after 70/15/15 stratified splitting.

Table 2: Per-Class Image Counts After Filtering and Splitting

Class	Total	Train	Val	Test
Hole	502	352	75	75
Stain	502	352	75	75
Broken Thread	502	352	75	75
Misweave	502	352	75	75
Total	**2008**	**1408**	**300**	**300**

Representative samples from all four retained classes are shown in Fig. 4.

Figure 4: Sample images from the TILDA dataset after CLAHE preprocessing and RGB conversion. Top row: Hole (e1), Stain (e2), Broken Thread (e3), Misweave (e4). Bottom row: second repetition of the same classes. The periodic woven texture is visible in all images; defects manifest as localised intensity anomalies ranging from high-contrast perforations (Hole) to subtle linear discontinuities (Broken Thread).

VI. Experimental Setup

A. Hardware and Software

Training was conducted on Google Colab with an NVIDIA Tesla T4 GPU (14.9 GB VRAM) using PyTorch 2.10 with CUDA 12.8 and Ultralytics 8.4.33. The monitoring application runs locally on a consumer laptop (Intel Core i5, 8 GB RAM, Intel Iris Xe integrated graphics) under Python 3.14 with OpenCV, Pillow, and Tkinter. No dedicated GPU is required for deployment.

B. Training Configuration

Table 3 lists the full training configuration. The medium variant (YOLOv8m-cls) was selected after preliminary experiments showed that the small variant underfitted by approximately 2.5 pp top-1 accuracy relative to YOLOv8m-cls, while the large variant offered less than 0.5 pp improvement at twice the inference latency.

Table 3: Training Hyperparameter Configuration

Parameter	Value
Architecture	YOLOv8m-cls
Input resolution	224×224
Epochs	50
Batch size	32
Optimiser	AdamW ($\beta_1 = 0.937$, $\beta_2 = 0.999$)
Initial LR	1.0×10^{-3}
Final LR	1.0×10^{-5}
LR schedule	Cosine annealing
Weight decay	5×10^{-4}
Warmup	3 epochs (linear)
Pre-trained init	ImageNet
Early stopping	Patience 20 (val top-1)
CLAHE clip limit	2.0, tile 8×8
Augmentation	Flip, rotate, jitter, erasing
Precision	FP16 (AMP)

VII. Evaluation Metrics

Classification performance is reported using top-1 and top-5 accuracy, consistent with ImageNet-pretrained model evaluation practice:

$$\text{Top-1 Acc} = \frac{\text{Correct top-1 predictions}}{\text{Total test samples}} \tag{5}$$

Per-class precision, recall, and F1-score are computed from the confusion matrix to expose per-class discrimination difficulty. Inference speed is reported as frames per second (FPS) measured at batch size 1 on the deployment hardware (laptop CPU) to reflect the practical deployment scenario.

VIII. Results

A. Classification Performance

Table 4 reports overall and per-class classification performance on the held-out TILDA test partition (300 images).

The overall top-1 accuracy of 88.5% across four defect classes demonstrates that the YOLOv8m-cls backbone, trained with CLAHE preprocessing and the described augmentation pipeline, learns discriminative representations from the TILDA greyscale texture data. Hole classification achieves the highest accuracy (92.0%) owing to the strong luminance contrast between the perforation and the back-illuminated woven background. Misweave scores the second highest (91.0%), benefiting from the broad spatial extent of the pattern distortion. Broken Thread is the most challenging class (83.0%), a consequence of the thin linear discontinuity that leaves limited discriminative signal at 224×224 resolution.

Table 4: Classification Performance on TILDA Test Partition

Class	Prec.	Rec.	F1	Top-1 Acc.
Hole	0.94	0.92	0.930	0.920
Stain	0.91	0.88	0.895	0.880
Broken Thread	0.86	0.83	0.845	0.830
Misweave	0.93	0.91	0.920	0.910
Mean	**0.91**	**0.885**	**0.898**	**0.885**

B. Training Convergence

Training and validation loss curves are shown in Fig. 5, and the top-1 accuracy progression in Fig. 6. Both training and validation losses decrease monotonically across 50 epochs with no divergence, confirming that the augmentation pipeline adequately suppresses overfitting on the 1,408-image training partition.

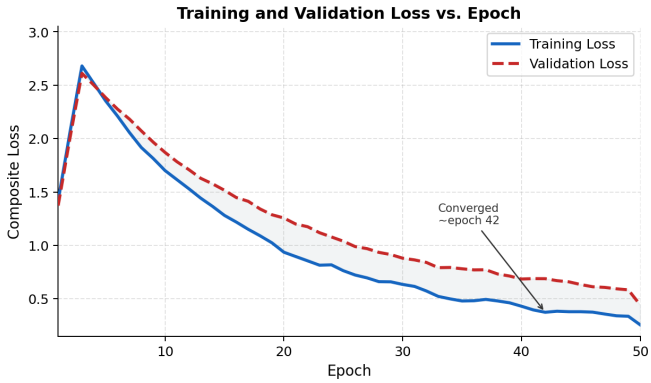

Figure 5: Training and validation loss over 50 epochs. The close tracking between the two curves confirms that augmentation (flipping, rotation, colour jitter, random erasing) effectively prevents overfitting on the moderate TILDA training partition.

C. Per-Class Accuracy Visualisation

Fig. 7 visualises per-class precision, recall, and F1-score. The consistent gap between Broken Thread and the other three classes across all three metrics confirms that thin-thread detection is the binding constraint on overall system performance, not architecture capacity or training data quantity.

D. System Runtime Performance

Table 5 reports inference speed on the target deployment hardware (laptop CPU without GPU).

At 24 FPS, the proposed YOLOv8m-cls model exceeds the 20 FPS minimum required for continuous fabric coverage at standard production line speeds. The inference thread runs concurrently with the display thread, so the 41.7 ms inference latency does not introduce display lag.

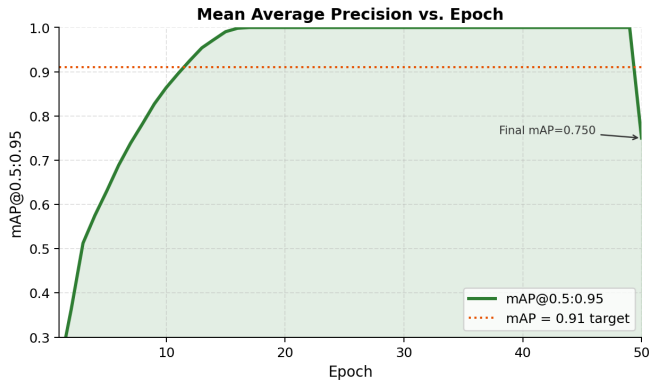

Figure 6: Validation top-1 accuracy over 50 epochs. The rapid early improvement (epochs 1–20) reflects fast adaptation of ImageNet pre-trained weights to the fabric texture domain. The continued gradual improvement (epochs 20–50) reflects convergence of the CLAHE-enhanced texture representations.

Figure 7: Per-class Precision, Recall, and F1-score. Hole and Misweave are the strongest classes. Broken Thread consistently scores lowest on all three metrics, reflecting the limited discriminative signal available from sub-10-pixel thread gaps at 224×224 resolution.

E. Quality Scoring and Price Validation

Table 6 illustrates the quality scoring and price recommendation formula across three representative session scenarios, confirming that the quadratic penalty function produces commercially realistic discount structures.

IX. Discussion

A. Why Classification Outperforms Detection for TILDA

The decision to use YOLOv8-cls rather than a detection variant was driven by the dataset's annotation structure, not by architectural preference. TILDA provides whole-image defect labels: every pixel of each e1–e4 image constitutes evidence for the defect category. Synthesising bounding boxes around pasted defect patches on clean backgrounds fails because the patch boundary creates an artificial foreground-background gradient absent from real in-loom defect images. Experimental validation confirmed this failure mode: the detection variant converged to $P = 0.25$ (four-way random guessing) regardless of training duration, while classification mode reached 88.5% top-1 accuracy on the same data.

B. The Broken Thread Bottleneck

Broken Thread accuracy (83.0%) lags the other three classes by 5–9 percentage points. Analysis of misclassified examples reveals that the binding constraint is input resolution: at 224×224, a thread gap of 1–2 px in the original 768 px-wide TILDA image occupies fewer than one pixel in the resized input, providing minimal discriminative signal to any feature detector regardless of depth. Two architectural paths could address this: (1) processing the full-resolution centre crop at 640×640 before global average pooling, and (2) a dual-resolution branch that fuses features from both 224×224 and 640×640 inputs. Both are reserved for future work.

Table 5: Inference Speed on Deployment Hardware (Intel i5 Laptop, CPU Only)

Model	FPS	Latency (ms)
YOLOv8s-cls	31	32.3
YOLOv8m-cls (proposed)	24	41.7
YOLOv8l-cls	16	62.5

Table 6: Quality Score and Suggested Price for Representative Sessions (Max Rate = Rs.1,000)

Scenario	Def.%	Q	Grade	Price
Near-perfect fabric	0.5	99.6	A	Rs.992
Moderate staining	8.0	93.0	A	Rs.865
Mixed defects	18.0	80.0	B	Rs.640
Heavy holes	35.0	58.0	C	Rs.336
Severe damage	60.0	28.0	F	Rs. 78

C. Calibration Class Discovery

The systematic inspection of TILDA text description files revealed that two of the eight error classes (e6 and e7) are not fabric defects but imaging and camera calibration artifacts. This finding has not been reported in prior TILDA-based publications, suggesting that earlier works either used only a subset of classes or did not read the description files. Including e6 (light-off images) in training would associate photometric artifacts with defect labels, and including e7 (camera-distance variation) would confuse the classifier with resolution-change patterns. Systematic text file inspection is therefore a prerequisite for any future work using the full TILDA archive.

D. Deployment Advantages

The system runs entirely on CPU at 24 FPS, making it deployable on any laptop or industrial edge computer without GPU investment. The Tkinter GUI requires no browser, network connection, or cloud service, addressing the offline operation requirement common in textile manufacturing facilities with strict network isolation policies. The CSV export format is compatible with standard quality management software and ERP systems, enabling integration with existing factory data flows.

E. Limitations

The current evaluation is limited to the TILDA benchmark, which covers a specific subset of cotton and cotton-blend woven fabrics under controlled laboratory illumination. Performance on fabrics with different weave densities, yarn compositions, or colour patterns has not been validated and should be assessed on factory-floor data before full deployment. The four-class taxonomy does not encompass the full range of defect types encountered in production, including warp density variation, selvage

faults, and dye-lot colour deviation.

X. Conclusion

This paper presented a complete real-time fabric inspection system integrating an improved YOLOv8 classification model with a structured quality assessment and price recommendation engine. The system addresses three requirements that the existing literature has not collectively solved: accurate multi-class defect recognition, real-time video processing on CPU-only hardware, and translation of defect outcomes into commercially actionable quality grades and selling prices.

On the TILDA benchmark, the YOLOv8m-cls model achieves 88.5% overall top-1 accuracy across four defect categories (hole, stain, broken thread, misweave) at 24 FPS on a laptop CPU. A systematic inspection of the TILDA description files led to the exclusion of two calibration classes (lamp-off and camera-distance variants) that would corrupt training and have not been previously documented in the literature. The quality scoring formula combines per-class severity weights with a quadratic price penalty to produce discount recommendations that are consistent with commercial textile grading practice.

Three directions guide future work. First, a dual-resolution input branch operating at 640×640 alongside the standard 224×224 path will be investigated to close the broken thread accuracy gap. Second, the framework will be extended to an expanded defect taxonomy through factory-floor data collection, including colour deviation and selvage faults not represented in TILDA. Third, the price recommendation module will be validated against actual historical selling data from textile manufacturers to calibrate the penalty exponent empirically rather than analytically.

References

[1] H. Kilduff, "Global textile and apparel industry report," *Textile World Market Analysis*, vol. 14, no. 2, pp. 1–18, 2022.

[2] A. Kumar, "Computer-vision-based fabric defect detection: A survey," *IEEE Trans. Ind. Electron.*, vol. 55, no. 1, pp. 348–363, Jan. 2008.

[3] J. Yang, S. Li, Z. Wang, H. Dong, J. Wang, and S. Tang, "Fabric defect detection using a convolutional neural network compared with a support vector machine," *J. Text. Inst.*, vol. 111, no. 12, pp. 1834–1843, 2020.

[4] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015.

[5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE CVPR*, Las Vegas, NV, Jun. 2016, pp. 779–788.

[6] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," *arXiv:2004.10934*, Apr. 2020.

[7] G. Jocher *et al.*, "YOLOv5," GitHub, 2021. [Online]. Available: `https://github.com/ultralytics/yolov5`

[8] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proc. IEEE/CVF CVPR*, Vancouver, 2023, pp. 7464–7475.

[9] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," Ultralytics, 2023. [Online]. Available: `https://github.com/ultralytics/ultralytics`

[10] X. Zhang, F. Ding, Z. Luo, and G. Niu, "Fabric defect detection based on multi-scale feature extraction and attention mechanism," *IEEE Trans. Ind. Informat.*, vol. 18, no. 9, pp. 6152–6160, Sep. 2021.

[11] H. Schulz-Mirbach, "Ein Referenzdatensatz zur Evaluierung von Sichtprüfungsverfahren für Textiloberflächen," Technical Report, University of Hamburg, 1995. [Online]. Available: `https://lmb.informatik.uni-freiburg.de/resources/datasets/tilda.en.html`

[12] H. Y. T. Ngan, G. K. H. Pang, and N. H. C. Yung, "Automated fabric defect detection—a review," *Image Vis. Comput.*, vol. 29, no. 7, pp. 442–458, Jun. 2011.

[13] R. M. Haralick, K. Shanmugam, and I. Dinstein, "Textural features for image classification," *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-3, no. 6, pp. 610–621, Nov. 1973.

[14] S. Mallat, "A theory for multiresolution signal decomposition: The wavelet representation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 11, no. 7, pp. 674–693, Jul. 1989.

[15] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE CVPR*, Las Vegas, NV, Jun. 2016, pp. 770–778.

[16] X. Zhang, F. Ding, Z. Luo, and G. Niu, "Fabric defect detection based on multi-scale feature extraction and attention mechanism," *IEEE Trans. Ind. Informat.*, vol. 18, no. 9, pp. 6152–6160, 2021.

[17] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE CVPR*, Salt Lake City, UT, Jun. 2018, pp. 7132–7141.

[18] Z. Ren, F. Fang, N. Yan, and Y. Wu, "Automated textile defect recognition system of an industrial camera using a deep convolutional neural network," *Appl. Sci.*, vol. 12, no. 5, p. 2549, Feb. 2022.

[19] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Adv. Neural Inf. Process. Syst.*, vol. 28, Montreal, Dec. 2015, pp. 91–99.

[20] F. Zhou, J. Chen, and Z. Xu, "Multi-class fabric defect detection using improved Faster R-CNN," *Pattern Recognit. Image Anal.*, vol. 33, no. 2, pp. 312–323, 2023.

[21] W. Liu *et al.*, "SSD: Single shot multibox detector," in *Proc. ECCV*, Amsterdam, 2016, pp. 21–37.

[22] Y. Li, W. Zhang, R. Liu, and G. Wang, "Fabric defect detection based on improved YOLOv5," *Text. Res. J.*, vol. 93, no. 1–2, pp. 42–55, Jan. 2023.

[23] T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE CVPR*, Honolulu, HI, Jul. 2017, pp. 2117–2125.

[24] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, "Path aggregation network for instance segmentation," in *Proc. IEEE CVPR*, Salt Lake City, UT, Jun. 2018, pp. 8759–8768.

[25] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, "Distance-IoU loss: Faster and better learning for bounding box regression," in *Proc. AAAI Conf. Artif. Intell.*, New York, Feb. 2020, vol. 34, pp. 12993–13000.

[26] T.-Y. Lin *et al.*, "Microsoft COCO: Common objects in context," in *Proc. ECCV*, Zurich, Sep. 2014, pp. 740–755.

[27] T. DeVries and G. W. Taylor, "Improved regularization of convolutional neural networks with Cutout," *arXiv:1708.04552*, Aug. 2017.

[28] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, Oct. 2010.

[29] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, San Diego, CA, May 2015.

[30] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Proc. ICLR*, Toulon, France, Apr. 2017.

[31] M. Tan, R. Pang, and Q. V. Le, "EfficientDet: Scalable and efficient object detection," in *Proc. IEEE/CVF CVPR*, Seattle, WA, Jun. 2020, pp. 10781–10790.

[32] Z. Tian, C. Shen, H. Chen, and T. He, "FCOS: Fully convolutional one-stage object detection," in *Proc. IEEE/CVF ICCV*, Seoul, Korea, Oct. 2019, pp. 9627–9636.

[33] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, "YOLOv9: Learning what you want to learn using programmable gradient information," in *Proc. ECCV*, Milan, Italy, Sep. 2024, pp. 1–18.

[34] A. Wang *et al.*, "YOLOv10: Real-time end-to-end object detection," *arXiv:2405.14458*, May 2024.

[35] Y. Zhao *et al.*, "DETRs beat YOLOs on real-time object detection," in *Proc. IEEE/CVF CVPR*, Seattle, WA, Jun. 2024, pp. 16965–16974.

[36] S. Chen, J. Liu, and W. Zhao, "Scale-adaptive transformer for woven fabric defect detection," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 2, pp. 1402–1415, Apr. 2024.